Research Project Proposal for FIT2082

1 Introduction

1.1 Background

Web application vulnerability scanners such as OWASP ZAP, Burp, Nessus, and more are a standard part of web security testing, but they typically return long, unranked lists of vulnerability findings rather than an actionable priority order [1]. So, security teams are then left to manually decide what to fix first, a process that consumes time which could otherwise be spent on remediation itself; the longer a critical vulnerability sits unranked in a long list, the longer it remains unpatched and exploitable in production.

1.2 Problem Statement & Motivation

The lack of prioritisation is worsened by the inconsistencies across scanners: ZAP supports only manual risk scoring with limited automation, unlike tools such as Acunetix which rank findings using CVSS directly [2][3]. ZAP reports its own Risk/Confidence ratings mapped to CWE categories, Nessus reports CVSS natively, and Burp has its own model, so a single uniform ranking rule cannot be applied across scanners without extra work.

This project addresses that gap directly by building a lightweight Python tool that ingests raw scanner output, applying a justified scoring model, and producing a clear, prioritised “fix-first” list allowing for not only ease-of-use …

1.3 Ethical Considerations

It is illegal to scan application without explicit permission from their owners, so this project will only scan its own test application.

Use open source/online application as much as possible …

2 Aims and Research Questions

The main aim of this project is to design and implement a lightweight tool in Python that converts raw, unranked vulnerability scanner output into a single, prioritised “fix-first” list.

This aim is broken down into the following research questions:

- How can findings from scanners with fundamentally different severity models (e.g. ZAP’s Risk/Confidence, Nessus’s native CVSS, Burp’s model) be normalised onto one comparable scale?

- What scoring or ranking model/heuristic produces a prioritised list that is both accurate and consistent when applied across multiple scanners?
- Does extending the tool to multiple scanners preserve consistent rankings for the same underlying vulnerabilities, or does cross-scanner disagreements remain after normalisation?

2.1 Objectives

2.1.1 In-Scope

- Select a primary vulnerability scanner (E.g. OWASP ZAP) and set up a vulnerable practice application (Juice Shop/DVWA) for testing.
- Building a Python parser the converts raw scanner output into a normalised finding model: Id, Name, CWE, Severity, Location
- Design and justify a severity scoring/ranking model to prioritise findings
- Produce a clean, sorted "fix-first" output (CLI + Report)
- Extend the tool to support multiple scanners (E.g. Burp, Nessus, etc.)
- Evaluate the Python tool by producing a before/after prioritised list for the test application (On the Shared GitHub Repository)

2.1.2 Out-Scope

- Risk assessment or other higher complexity prioritisation methods beyond the chosen scoring model (e.g. EPSS-style exploit prediction, etc.)
- Any features beyond parsing, scoring, and reporting (e.g. automated remediation, etc.)

2.2 Expected Outcome & Evaluation

The expected outcome is a functional Python tool that produces a correctly prioritised "fix-first" list for a single scanner, then extends to support multiple scanners (>=2).

Evaluation will cross-validate rankings against a fixed set of known vulnerability classes seeded in the practice application (e.g. OWASP Top 10, 2025, which includes Broken Access Controls, Security Misconfigurations, Injection, etc.) checking that each vulnerability is assigned a consistent rank position regardless of which scanner detected it.

3 Methodology

The project follows an iterative approach: Scanner Report -> Parse -> Rank -> Prioritised List. Development will proceed end-to-end on a single scanner before a second scanner is introduced to minimise scope creep.

3.1 Responsibilities & Roles

1. Environment & Data Collection: Set up a vulnerability scanner against a deliberately vulnerable practice application and capture sample scan reports in their raw output format.
2. Parsing: Build a Python parser that reads raw scanner output and converts it into a normalised finding model (Id, Name, CWE, Severity, Location), resolving the format mismatches between different vulnerability scanners.
3. Scoring & Ranking: Design and justify a severity scoring model then normalise each scanner's native severity to a common scale mapping to CVSS/heuristic equivalents.
4. Extension & Evaluation: Extend the parser and scoring model to a second scanner then cross-validate rankings against known vulnerability classes seeded in the test application to check for rank consistency across scanners.

Both students must be able to explain the whole pipeline end-to-end.

There are two roles: one role deals with Environment, Data Collection and Parsing while the other role deals with Scoring, Ranking, Extension and Evaluation.

Rhythm:

- Weekly 30-45 Minute Meetings
- Shared Log of Decisions, Actions, and Blockers
- One Shared GitHub Repository
- Rotating Which Student Presents Each Week

3.2 Risks & Mitigation

3.2.1 Risks

1. Scanners disagree on severity due to different scoring perspectives which produces inconsistent rankings for the same vulnerability class.
2. Scope creep from attempting to handle multiple scanners before the first pipeline works end-to-end.
3. CVSS mapping/scoring-model design proves harder than expected which greatly slows down progress.

3.2.2 Mitigation

1. Choose and justify one normalisation strategy early and validate it against known vulnerability classes.
2. Get one scanner working fully (parse -> rank -> output) before adding a second.
3. Fall back to a simpler heuristic mapping if a more rigorous model is not achievable in time.

3.3 Justification

A single-scanner approach was chosen to reduce integration risk and make the scoring-model errors easier to isolate and debug. Parsing is developed first to establish a normalised finding model before ranking logic is added. The scoring model maps each scanner's native severity onto a common CVSS/heuristic scale since CVSS is the industry standard for comparable and justified severity ranking [1]. Cross-validation against known vulnerability classes allows for confirmation that the normalisation produces consistent rankings across scanners.

4 Timeline

Phase	Dates	Milestone
Setup & proposal	10–28 Aug	Proposal signed off; env + repo live
Data & parsing	31 Aug–11 Sep	Parser reads reports
Scoring & prioritisation	14–25 Sep	Walking skeleton working
Output & reporting	28 Sep–2 Oct	Sorted CLI output + report
Buffer / stretch	5–9 Oct	2nd scanner *if on track*
Evaluation	12–16 Oct	Before/after list produced
Write-up & polish	19–23 Oct	Draft report for feedback
Finalise	26–30 Oct	Revisions + demo rehearsal
Submission	**2 Nov**	**Report + tool + demo**

5 Bibliography

[1] B, A., W, M., N, M., O, J., & S, S. (2025). *Automatic CVSS-Based Vulnerability Prioritization and Response with Context Information and Machine Learning*. https://doi.org/10.3390/app15168787

[2] B, S., NRK, S., J, T., & S, S. (2024). *A Comparative Analysis of Vulnerability Management Tools: Evaluating Nessus, Acunetix, and Nikto for Risk Based Security Solutions*. https://doi.org/10.48550/arxiv.2411.19123

[3] E, A., E, A.-S., W, S., & X, N. (2023). *Automated CVE Analysis for Threat Prioritization and Impact Prediction*. https://doi.org/10.48550/arxiv.2309.03040